Placement Management System

Project Report

Course: UCS310 — Database Management Systems
Institution: Thapar Institute of Engineering and Technology, Patiala
Team Lead: Shivam Raj
Date: May 2026

Table of Contents

1. Introduction
2. Objectives
3. System Requirements
4. System Architecture
5. Database Design
6. ER Diagram
7. Relational Schema (3NF)
8. SQL Implementation
9. Application Design
10. User Interface
11. Testing & Results
12. Conclusion
13. References

---

1. Introduction

1.1 Background

Campus placement is a critical phase in the academic lifecycle of engineering students. Managing the end-to-end placement process — from company registrations and eligibility filtering to application tracking and result announcements — involves significant administrative overhead. Traditional manual methods using spreadsheets and emails are error-prone, lack real-time visibility, and do not scale as the number of students and recruiting companies grows.

1.2 Problem Statement

Design and implement a web-based **Placement Management System (PMS)** that automates the complete placement workflow for an engineering institution. The system must demonstrate mastery of relational database concepts including normalization, stored procedures, triggers, joins, and aggregate functions — all core topics of the UCS310 DBMS curriculum.

1.3 Scope

The system covers the following functional areas:

- **Student Management** — Registration, profile maintenance, and placement status tracking.
- **Company Management** — CRUD operations for recruiting companies with configurable eligibility criteria.
- **Application Workflow** — Students apply to eligible companies; administrators track application statuses.
- **Result Recording** — Admin records final outcomes; the database automatically updates placement statuses.
- **Reporting & Analytics** — Company-wise placement reports, eligibility lists, and aggregate statistics.

---

2. Objectives

#	Objective	DBMS Concept Demonstrated
1	Design a relational schema in Third Normal Form (3NF)	Normalization, Functional Dependencies
2	Implement stored procedures for eligibility checks and application processing	Stored Procedures, Parameterized Queries
3	Use triggers for automatic status management	Database Triggers, Event-Driven Logic
4	Generate reports using multi-table JOINs and aggregate functions	JOINs, GROUP BY, Aggregate Functions
5	Perform full CRUD operations through a web interface	SQL DML (INSERT, SELECT, UPDATE, DELETE)
6	Enforce data integrity through constraints and validation	CHECK, UNIQUE, FOREIGN KEY, NOT NULL
7	Implement role-based access control	Authentication, Authorization

---

3. System Requirements

3.1 Software Requirements

Component	Technology	Version
Programming Language	Python	3.12+
Web Framework	Flask	3.1.1
ORM	Flask-SQLAlchemy	3.1.1
Database (Production)	MySQL	8.0+
Database (Development)	SQLite	3.x
Authentication	Flask-Login	0.6.3
Migration	Flask-Migrate (Alembic)	4.1.0

Component	Technology	Version
DB Connector	PyMySQL	1.1.1
Password Hashing	Werkzeug Security	3.1.3

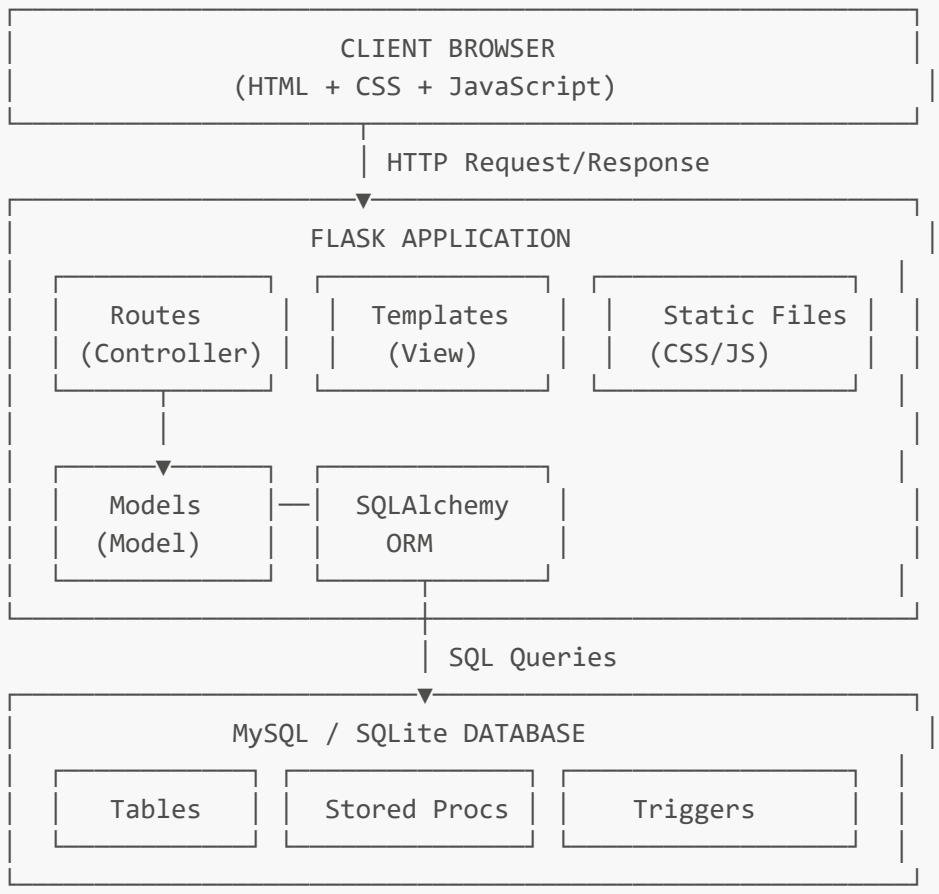
3.2 Hardware Requirements

Component	Minimum Specification
Processor	Intel i3 / equivalent
RAM	4 GB
Storage	500 MB free space
Network	Required for font loading (Google Fonts)

---

4. System Architecture

4.1 Architecture Pattern — MVC (Model-View-Controller)



4.2 Directory Structure

```
placement management system/
├── run.py                          # Application entry point
├── requirements.txt                # Python dependencies
├── app/
│   ├── __init__.py                 # App factory + extensions
│   ├── config.py                   # Configuration (DB URI, secrets)
│   ├── models.py                   # SQLAlchemy ORM models
│   ├── routes/
│   │   ├── __init__.py
│   │   ├── auth.py                 # Login, Register, Logout
│   │   ├── admin.py                # Admin CRUD operations
│   │   ├── student.py              # Student dashboard & applications
│   │   └── reports.py              # Placement reports & eligibility
│   ├── templates/                  # Jinja2 HTML templates
│   │   ├── base.html               # Base layout with sidebar
│   │   ├── auth/                   # login.html, register.html
│   │   ├── admin/                  # dashboard, students, companies, etc.
│   │   ├── student/                # dashboard, companies, applications
│   │   └── reports/                # placement_report, eligibility_list
│   └── static/
│       ├── css/style.css           # Complete stylesheet
│       └── js/main.js              # Client-side scripts
└── sql/
    ├── 01_schema.sql               # DDL — Tables & Constraints
    ├── 02_procedures.sql           # Stored Procedures
    ├── 03_triggers.sql             # Database Triggers
    └── 04_seed.sql                 # Sample/Seed Data
```

---

5. Database Design

5.1 Entity Identification

Entity	Description	Key Attributes
User	Authentication record for all system users	id, username, email, password_hash, role
Student	Academic profile of a registered student	id, user_id (FK), roll_no, name, branch, cgpa, backlogs, placement_status
Company	Recruiting company with eligibility criteria	id, name, role_offered, package_lpa, min_cgpa, max_backlogs, allowed_branches
Application	A student's application to a company	id, student_id (FK), company_id (FK), status
Result	Final placement outcome of an application	id, application_id (FK), result, offer_date, remarks

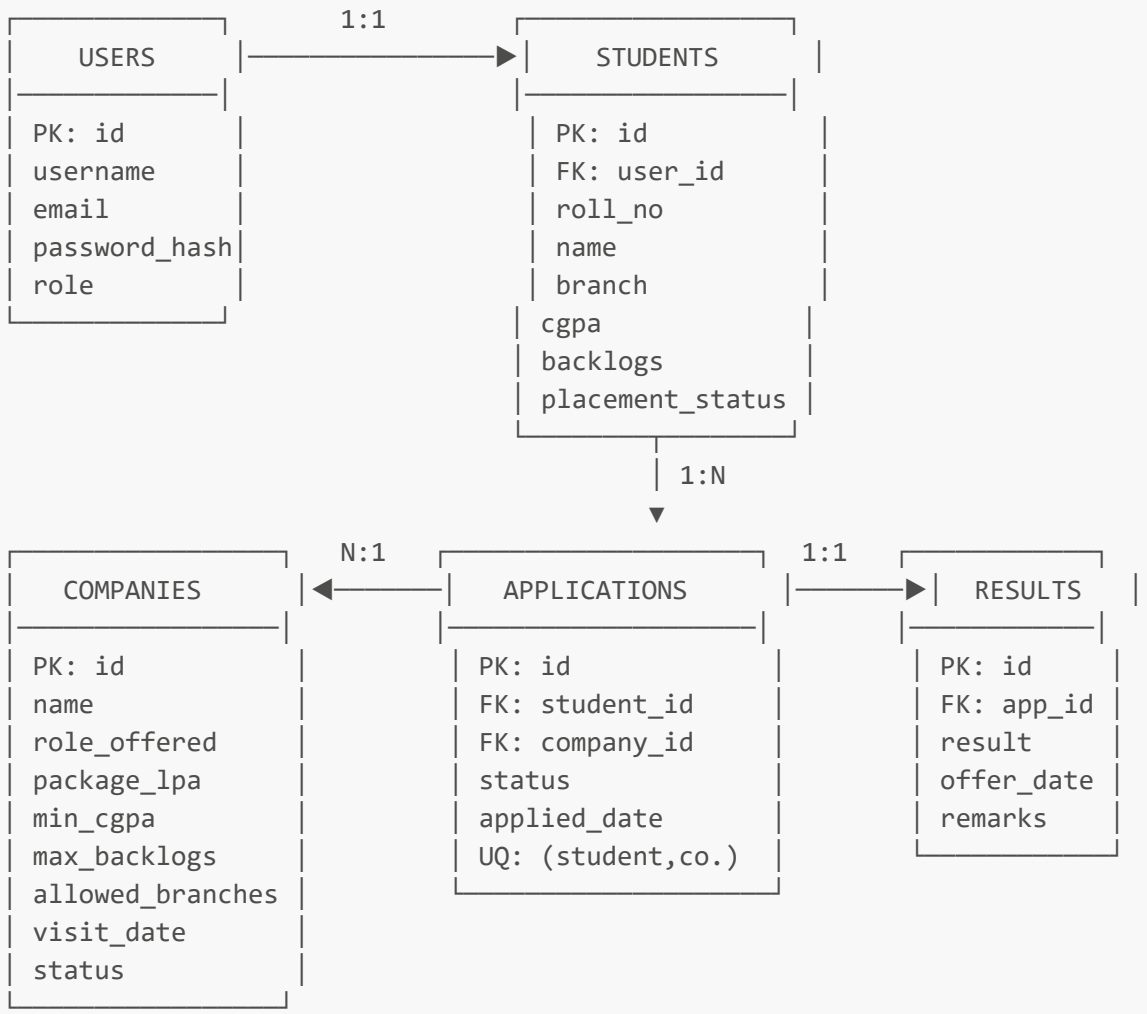
5.2 Relationships

Relationship	Type	Description
User → Student	1:1	Each student user has exactly one profile
Student → Application	1:N	A student can apply to many companies
Company → Application	1:N	A company receives many applications
Application → Result	1:1	Each application has at most one result

The **Application** table serves as a **bridge entity** resolving the Many-to-Many relationship between Students and Companies.

---

6. ER Diagram



---

7. Relational Schema (Third Normal Form)

7.1 Table Definitions

Table: users

```
users (
    id           INT PRIMARY KEY AUTO_INCREMENT,
    username     VARCHAR(50) NOT NULL UNIQUE,
    email        VARCHAR(120) NOT NULL UNIQUE,
    password_hash VARCHAR(256) NOT NULL,
    role         ENUM('admin','student') DEFAULT 'student',
    created_at   TIMESTAMP,
    updated_at   TIMESTAMP
)
```

Table: students

```
students (
    id               INT PRIMARY KEY AUTO_INCREMENT,
    user_id          INT NOT NULL UNIQUE → users(id),
    roll_no          VARCHAR(20) NOT NULL UNIQUE,
    name             VARCHAR(100) NOT NULL,
    email            VARCHAR(120) NOT NULL,
    phone            VARCHAR(15),
    branch           ENUM('CSE','ECE','ME','CE','EE','CHE') NOT NULL,
    cgpa             DECIMAL(4,2) NOT NULL  CHECK(0.00–10.00),
    backlogs         INT NOT NULL DEFAULT 0 CHECK(≥0),
    placement_status ENUM('Unplaced','Placed') DEFAULT 'Unplaced',
    created_at       TIMESTAMP,
    updated_at       TIMESTAMP
)
```

Table: companies

```
companies (
    id               INT PRIMARY KEY AUTO_INCREMENT,
    name             VARCHAR(150) NOT NULL,
    description      TEXT,
    role_offered     VARCHAR(100) NOT NULL,
    package_lpa      DECIMAL(6,2) NOT NULL CHECK(>0),
    min_cgpa         DECIMAL(4,2) DEFAULT 0.00 CHECK(0.00–10.00),
    max_backlogs     INT DEFAULT 0 CHECK(≥0),
    allowed_branches VARCHAR(100) DEFAULT 'CSE,ECE,ME,CE,EE,CHE',
    visit_date       DATE,
    status           ENUM('Upcoming','Ongoing','Completed') DEFAULT 'Upcoming',
    created_at       TIMESTAMP,
    updated_at       TIMESTAMP
)
```

Table: applications

```
applications (
    id           INT PRIMARY KEY AUTO_INCREMENT,
    student_id   INT NOT NULL → students(id),
    company_id   INT NOT NULL → companies(id),
    applied_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    status       ENUM('Applied','Shortlisted','Interview','Offered','Rejected')
                 DEFAULT 'Applied',
    UNIQUE(student_id, company_id)
)
```

Table: results

```
results (
    id              INT PRIMARY KEY AUTO_INCREMENT,
    application_id  INT NOT NULL UNIQUE → applications(id),
    result          ENUM('Selected','Not Selected') NOT NULL,
    offer_date      DATE,
    remarks         TEXT,
    created_at      TIMESTAMP
)
```

7.2 Normalization Justification (3NF)

Normal Form	Justification
1NF	All columns hold atomic values. No repeating groups or multi-valued attributes.
2NF	Every non-key attribute depends on the full primary key. All PKs are single-column AUTO_INCREMENT, so partial dependencies are impossible.
3NF	No transitive dependencies exist. Each non-key attribute depends directly on the primary key of its table. `placement_status` in `students` is a controlled denormalization maintained automatically by a database trigger.

7.3 Integrity Constraints

Constraint Type	Example	Purpose
PRIMARY KEY	`students.id`	Unique row identification
FOREIGN KEY	`students.user_id → users.id`	Referential integrity
UNIQUE	`applications(student_id, company_id)`	Prevent duplicate applications
CHECK	`cgpa BETWEEN 0.00 AND 10.00`	Domain validation
NOT NULL	`students.name`	Mandatory fields

Constraint Type	Example	Purpose
ON DELETE CASCADE	All foreign keys	Automatic cleanup of dependent rows

8. SQL Implementation

8.1 Stored Procedures

Six stored procedures handle core business logic at the database level:

Procedure 1: `check_eligibility`

Purpose: Determines if a student is eligible for a specific company.

```
CREATE PROCEDURE check_eligibility(
    IN p_student_id INT,
    IN p_company_id INT,
    OUT p_eligible  TINYINT
)
-- Checks: CGPA >= min_cgpa, backlogs <= max_backlogs,
-- branch IN allowed_branches, placement_status = 'Unplaced'
```

Procedure 2: `apply_to_company`

Purpose: Validates eligibility, then inserts into the applications table.

```
CREATE PROCEDURE apply_to_company(
    IN p_student_id INT,
    IN p_company_id INT
)
-- Raises SQLSTATE 45000 if already applied or not eligible
-- Calls check_eligibility internally
```

Procedure 3: `get_eligible_companies`

Purpose: Returns all companies a student can apply to.

```
CREATE PROCEDURE get_eligible_companies(IN p_student_id INT)
-- Uses subquery to exclude already-applied companies
-- Ordered by package_lpa DESC
```

Procedure 4: `get_placement_report`

Purpose: Company-wise placement report using JOINs and aggregates.

```
CREATE PROCEDURE get_placement_report()
-- Uses: LEFT JOIN companies ⇔ applications ⇔ results
-- Aggregates: COUNT(), SUM(CASE WHEN ...) for selected/rejected/pending
-- GROUP BY company
```

Procedure 5: `get_eligibility_list`

Purpose: Lists all eligible students for a given company.

```
CREATE PROCEDURE get_eligibility_list(IN p_company_id INT)
-- Filters by: cgpa, backlogs, branch, placement_status
-- Uses FIND_IN_SET() for branch matching
```

Procedure 6: `record_result`

Purpose: Records final placement outcome for an application.

```
CREATE PROCEDURE record_result(
    IN p_application_id INT,
    IN p_result VARCHAR(20),
    IN p_offer_date DATE,
    IN p_remarks TEXT
)
-- Validates no duplicate result exists
-- INSERT triggers automatic status updates via trigger
```

8.2 Database Triggers

Two triggers automate status management:

Trigger 1: `trg_after_result_insert` **(AFTER INSERT ON results)**

Purpose: Automatically updates application status and student placement status when a result is recorded.

```
-- If result = 'Selected':
--   applications.status → 'Offered'
--   students.placement_status → 'Placed'
-- If result = 'Not Selected':
--   applications.status → 'Rejected'
```

Trigger 2: `trg_before_application_insert` **(BEFORE INSERT ON applications)**

Purpose: Re-validates eligibility at the database level, acting as a safety net even if the application bypasses the stored procedure.

```
-- Validates: placement_status, CGPA, backlogs, branch
-- Raises SQLSTATE 45000 on any validation failure
```

8.3 Key SQL Queries Demonstrating JOINs and Aggregates

Multi-table JOIN for placement report:

```
SELECT
    c.name AS company_name,
    c.role_offered,
    c.package_lpa,
    COUNT(a.id) AS total_applications,
    SUM(CASE WHEN r.result='Selected' THEN 1 ELSE 0 END) AS selected,
    SUM(CASE WHEN r.result='Not Selected' THEN 1 ELSE 0 END) AS rejected,
    COUNT(a.id) - COUNT(r.id) AS pending
FROM companies c
LEFT JOIN applications a ON c.id = a.company_id
LEFT JOIN results r ON a.id = r.application_id
GROUP BY c.id, c.name, c.role_offered, c.package_lpa
ORDER BY c.name;
```

Subquery for eligible companies:

```
SELECT c.*
FROM companies c
WHERE c.min_cgpa <= v_cgpa
  AND c.max_backlogs >= v_backlogs
  AND FIND_IN_SET(v_branch, c.allowed_branches) > 0
  AND c.id NOT IN (
      SELECT company_id FROM applications WHERE student_id = p_student_id
  )
ORDER BY c.package_lpa DESC;
```

---

9. Application Design

9.1 Authentication & Authorization

The system implements role-based access control with two roles:

Role	Access Level	Key Capabilities
Admin	Full system access	CRUD on students, companies, applications, results; view all reports

Role	Access Level	Key Capabilities
Student	Self-service access	View eligible companies, apply, view own applications and profile

- Passwords are hashed using **scrypt** via `werkzeug.security.generate_password_hash()`.
- Session management is handled by **Flask-Login** with a 1-hour session lifetime.
- Route-level decorators (`@admin_required`, `@student_required`) enforce authorization.

9.2 Route Architecture (Blueprints)

Blueprint	URL Prefix	Routes	Description
`auth`	`/`	login, register, logout, index	Authentication flows
`admin`	`/admin`	dashboard, students CRUD, companies CRUD, applications, results	Administrative operations
`student`	`/student`	dashboard, companies, apply, my_applications, profile	Student self-service
`reports`	`/reports`	placement_report, eligibility_list	Reporting and analytics

9.3 Template Architecture

All templates extend a common `base.html` that provides:

- **Sidebar navigation** — Role-aware menu items with active state highlighting
- **Flash message system** — Color-coded success/error/warning/info notifications
- **Responsive layout** — Mobile-friendly with collapsible sidebar
- **Footer** — Course and institution information

9.4 Design System

The UI follows a **dark glassmorphism** design language:

Design Element	Implementation
Color Scheme	Dark background (#0f0f1a) with vibrant gradient accents
Typography	Inter (Google Fonts) with weights 300–800
Cards	Semi-transparent with backdrop blur and subtle borders
Animations	Hover transitions, slide-down flash messages
Responsiveness	CSS Grid + media queries for mobile/tablet/desktop

---

10. User Interface

10.1 Login Page

- Centered glassmorphism card with gradient heading
- Username/password form with validation
- Link to student registration
- Admin credentials hint for demo

10.2 Admin Dashboard

- **6 stat cards** showing: Total Students, Placed, Unplaced, Companies, Applications, Offers
- **Progress bar** visualizing placement percentage
- **Recent applications table** with status badges

10.3 Student Management (Admin)

- Sortable table listing all students with Roll No, Name, Branch, CGPA, Status
- Add/Edit forms with full validation
- Delete with confirmation

10.4 Company Management (Admin)

- Company listing with package, criteria, and visit date
- Add/Edit forms with multi-select branch checkboxes
- Eligibility list link for each company

10.5 Student Dashboard

- Welcome header with placement status badge
- 3 stat cards: Applications, Offers, Pending
- Profile summary: Roll No, Branch, CGPA, Backlogs
- Recent applications table

10.6 Companies Page (Student)

- Grid of eligible companies with role, package, and Apply button
- List of already-applied companies with current status

10.7 Reports

- **Placement Report:** Aggregate stats + company-wise breakdown table
- **Eligibility List:** Per-company list of eligible students sorted by CGPA

---

11. Testing & Results

11.1 Test Scenarios

#	Test Case	Expected Result	Status
1	Admin login with `admin/admin123`	Redirect to admin dashboard	✅ Pass

#	Test Case	Expected Result	Status
2	Student self-registration	User + Student profile created, redirect to login	✅ Pass
3	Student applies to eligible company	Application record created with status 'Applied'	✅ Pass
4	Student applies to ineligible company (low CGPA)	Error message, no record created	✅ Pass
5	Duplicate application attempt	Warning message, no duplicate created	✅ Pass
6	Already-placed student tries to apply	Blocked by trigger/validation	✅ Pass
7	Admin records 'Selected' result	Student status auto-updated to 'Placed' via trigger	✅ Pass
8	Admin records 'Not Selected' result	Application status updated to 'Rejected'	✅ Pass
9	Placement report generation	Correct aggregate counts via JOINs	✅ Pass
10	Eligibility list for company	Only qualified students shown	✅ Pass
11	Admin CRUD on students	Create, Read, Update, Delete all functional	✅ Pass
12	Admin CRUD on companies	Create, Read, Update, Delete all functional	✅ Pass

11.2 Sample Seed Data Used

10 Students across 6 branches (CSE, ECE, ME, CE, EE, CHE) with CGPAs ranging from 6.50 to 9.20.

5 Companies with varying eligibility criteria:

Company	Package (LPA)	Min CGPA	Max Backlogs	Branches
Google	45.00	8.50	0	CSE, ECE
Microsoft	38.00	8.00	0	CSE, ECE, EE
Infosys	6.50	6.00	2	All
Tata Motors	8.00	7.00	1	ME, CE, ECE
Goldman Sachs	32.00	8.50	0	CSE, ECE, EE

---

12. Conclusion

12.1 Summary

The Placement Management System successfully demonstrates the practical application of core DBMS concepts in a real-world scenario. The project achieves all seven stated objectives:

1. **3NF Schema** — Five normalized tables with proper functional dependencies.
2. **Stored Procedures** — Six procedures encapsulating business logic at the database level.
3. **Triggers** — Two triggers for automatic status management and eligibility validation.
4. **JOINs & Aggregates** — Multi-table queries for comprehensive placement reporting.
5. **CRUD Operations** — Full create, read, update, delete functionality via a web interface.
6. **Data Integrity** — CHECK, UNIQUE, FOREIGN KEY, and NOT NULL constraints throughout.
7. **Role-Based Access** — Separate admin and student interfaces with proper authorization.

12.2 DBMS Concepts Demonstrated

Concept	Where Used
DDL (CREATE, ALTER)	Schema definition (01_schema.sql)
DML (INSERT, UPDATE, DELETE, SELECT)	All CRUD routes
Normalization (3NF)	Schema design with justified denormalization
Stored Procedures	Eligibility checks, applications, reports
Triggers	Auto status updates on result recording
JOINs (INNER, LEFT)	Placement report, application views
Aggregate Functions (COUNT, SUM)	Dashboard statistics, reports
Subqueries	Eligible companies, pending applications
Constraints (PK, FK, UNIQUE, CHECK)	All tables
Indexes	Optimized queries on frequently filtered columns
Transaction Management	Commit/rollback in Flask routes
ENUM Types	Role, status, branch columns

12.3 Future Enhancements

- **Email Notifications** — Automated alerts for application status changes.
- **Resume Upload** — File upload feature for student resumes.
- **Interview Scheduling** — Calendar integration for interview slots.
- **Analytics Dashboard** — Charts and graphs using Chart.js or D3.js.
- **Export to PDF/Excel** — Downloadable reports.
- **Multi-institution Support** — Tenant-based architecture for multiple colleges.

---

13. References

1. Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson.

2. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7th ed.). McGraw-Hill.
3. Flask Documentation — https://flask.palletsprojects.com/
4. SQLAlchemy Documentation — https://docs.sqlalchemy.org/
5. MySQL 8.0 Reference Manual — https://dev.mysql.com/doc/refman/8.0/en/
6. Flask-Login Documentation — https://flask-login.readthedocs.io/
7. Werkzeug Security — https://werkzeug.palletsprojects.com/en/stable/utils/#module-werkzeug.security

---

Appendix A: How to Run the Project

Prerequisites

- Python 3.12 or higher installed

Steps

```
# 1. Navigate to project directory
cd "placement management system"

# 2. Install dependencies
pip install -r requirements.txt

# 3. Run the application
python run.py

# 4. Open browser
# Visit http://127.0.0.1:5000
```

Default Credentials

Role	Username	Password
Admin	admin	admin123
Student	Register via /register	

Using MySQL (Optional)

```
# Set environment variable before running
set DATABASE_URL=mysql+pymysql://root:password@localhost:3306/placement_db

# Run SQL scripts in order
mysql -u root -p < sql/01_schema.sql
mysql -u root -p < sql/02_procedures.sql
mysql -u root -p < sql/03_triggers.sql
mysql -u root -p < sql/04_seed.sql
```

Appendix B: Complete File Listing

File	Lines	Purpose
`sql/01_schema.sql`	150	DDL — 5 tables with constraints and indexes
`sql/02_procedures.sql`	220	6 stored procedures
`sql/03_triggers.sql`	106	2 database triggers
`sql/04_seed.sql`	94	Seed data (1 admin, 10 students, 5 companies)
`app/__init__.py`	74	Flask app factory with extension init
`app/config.py`	47	Database and session configuration
`app/models.py`	108	5 SQLAlchemy ORM models
`app/routes/auth.py`	149	Login, register, logout routes
`app/routes/admin.py`	363	Admin CRUD (students, companies, apps, results)
`app/routes/student.py`	127	Student dashboard, apply, profile
`app/routes/reports.py`	76	Placement report and eligibility list
`app/templates/base.html`	72	Base layout with sidebar navigation
`app/static/css/style.css`	270+	Complete dark-theme stylesheet
`app/static/js/main.js`	33	Sidebar toggle, flash auto-dismiss
`run.py`	8	Application entry point
`requirements.txt`	8	Python dependencies

Total: 16 source files, ~1,900 lines of code

End of Report